Data collection

Source of Data: https://samate.nist.gov/SARD/test-suites/112

1. Sources of Data

For our project, we obtained the dataset from Juliet Test Suite for C/C++ (v1.3).

Juliet Test Suite is a collection of public test cases provided by the National Institute of Standards and Technology (NIST), an American institute. It contains examples of vulnerable codes of various Common Weakness Enumeration (CWE) types.

Why this source?

These sources are official data provided by NIST, so they are highly reliable. Also, labels are required for each data because we will choose supervised method for ML. This source can provide appropriate data for ML model learning because folders and files are organized by vulnerable type. In addition, it can be a good learning data with labels because the good and bad functions according to the vulnerable type are separated. Since it provides over 100,000 files and over 400,000 functions not only in terms of data quality but also in terms of quantity, we decided that it could be sufficient training data. We selected this source as the source of our dataset with above reasons.

In addition, this project dealt with source files limited to C and C++ languages. The reason is that C and C++ have features that programmers have to perform memory management (pointer, buffer, heap, stack, etc.) That's why serious security vulnerabilities such as buffer overflow, use-after-free, and memory leakage often occur compared to other high-level languages such as python. Therefore, in this project, which aims to detect vulnerable codes, although only two languages are used as additional datasets, we decided that focusing on these two languages has great practical analysis value and can reflect real-world security issues as well as the project.

---

2. Methods of Data Collection

After selecting these sources, we used a Python script to unzip the dataset and collect the code files. First, files were automatically collected while recursively searching the folder with os.walk(). Only files with extensions .c, .cpp, and .h were included, while non-source files were excluded. For content criteria, only codes in which security vulnerabilities were intentionally

inserted in the source file were included, and example codes that simply performed normal operation were excluded from the analysis. For analysis, in order to extract the code in units of function, the source code was separated using Regex expression. In addition to the source code, the file name included vulnerable types and language information, so this was extracted as data. Python's Pandas and Regex Library were used to collect these data.

---

3. Relevance and Sufficiency

The core goal of this project is to detect and analyze vulnerable codes. Our chosen Juliet Test Suite is closely related to this project. This dataset contains numerous C/C++ source codes that deliberately insert various security flaws as a data for security research that is officially provided by NIST. Therefore, unlike common normal codes, it provides data optimized for learning and detecting real-world security vulnerability types.

In particular, Dataset includes a variety of major CWE types such as buffer overflow, format string vulnerability, and memory leak, which can sufficiently cover the vulnerability analysis topic that the project wants to address. This diversity helps to choose the machine learning model next step and model to learn comprehensively multiple vulnerability type, not just a single.

As a result, Juliet Test Suite is a suitable source of data for achieving the objectives of this project in both the relevance and the sufficiency of the data.

---

4. Challenges Faced

There were problems encountered in the data collection process of this project. First, Juliet Test Suite contained over 100,000 large-scale files, which caused considerable memory use and processing speed degradation in the process of reading them all around Python. In addition, a regular expression was used to separate the source code into function units, but there was a limit to accurately handling complex code structures such as overlapping braces or the distinction between function and variable declaration units. Finally, even within the same CWE type, the quality of the code was not uniform, so some functions were too short or lacked meaning, making them unsuitable for machine learning. For this reason, additional filtering and refining were required during the data preprocessing process.

---

5. How Challenges Were Addressed

Several methods have been applied to solve these problems. For the large dataset size and memory usage, we used `os.walk()` to read files sequentially instead of loading everything

at once. We also used `with open(..., errors="ignore")` to avoid interruptions from encoding errors. In addtion,we printed progress updates after reading a set number of files to monitor performance. For inconsistent code quality, we applied quality filtering in preprocessing, removing very short or meaningless functions based on a minimum token count. These solutions helped improve both the consistency and quality of the final dataset.